DECOMPOSITION & PLANNING LAB

Giving a New Agent on Your MCP Server the Ability to Break Down and Search Over Hard Requests

THE TASK

You're extending the same company, the same shared GitHub repo, the same database, and the same MCP server from the MCP Server Lab and the Memory & RAG Lab. This time the work centers on a different agent than the one you gave memory and retrieval to. Your MCP tools are scoped and narrow on purpose, single lookups, single writes, one clean call in, one clean result out, but some real requests in your company are not single-call requests. They're multi-step, ambiguous, sometimes contradictory, and they require deciding what to do before there's anything to retrieve or call. That's a planning problem, not a memory problem and not a retrieval problem, and it needs its own agent.

Find the genuine version of that problem inside your own company, a request some real person actually sends today that no single tool call or single LLM turn can safely resolve and build an agent around it that decomposes the request into a DAG of sub-tasks, plans how to solve the pieces that need real reasoning, and checks its own output before it ships. You are not starting a new project, and you are not touching the memory/RAG agent's code path. This is a new agent, reusing the same mcp_server/ and db/, sitting next to the agent you already built.

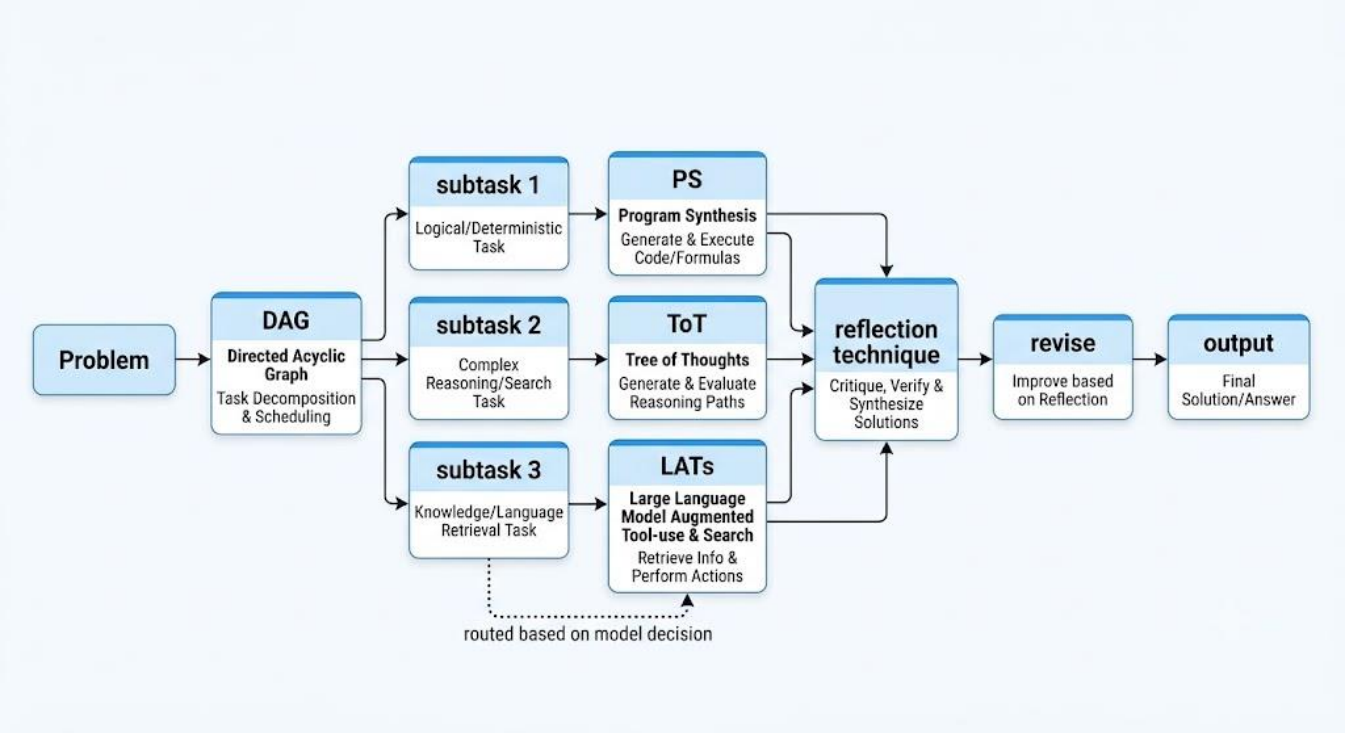
THE DECOMPOSITION & PLANNING CONCERNS

We covered a full decomposition-and-planning stack, and your system needs to demonstrate all of it, evaluated with real numbers, not just described. Your team must implement every concern below.

Design your extension so each concern has a genuine reason to exist in your system. A DAG built over three fake to-do items is worse than not having one. Pick a real recurring request that is actually too big, too ambiguous, or too failure-prone for a single call. Look at the chart at the end of this section to understand how the prompt should flow through the system.

- **Task decomposition into a DAG, both methods, not one:** implement decomposition-first (the whole plan is generated up front, in one shot, then executed in topological order) and dynamic/interleaved decomposition (the next sub-task is generated after observing the result of the last one, so an early surprise or failure can reshape what comes next) against the same real request type. Enforce acyclicity at construction time; a plan that can deadlock is a bug, not an edge case. Show a real case where the two methods diverge, where dynamic decomposition changes course after an early sub-task result that decomposition-first would have blindly executed anyway, but also show token usage and if the tests you wrote for your agent resulted in bigger context.
- **Planning algorithms, all three:** implement Plan-and-Solve (one explicit plan phase, then execute step by step, single pass, no branching), Tree of Thoughts (generate several candidate next-steps, self-evaluate each, search with BFS or DFS, keep or prune branches), and LATS (MCTS-guided search where branches are scored by real external feedback, not the model's own opinion of itself, and failed branches get a verbal reflection that steers where the search looks next). These are not interchangeable defaults, each sub-task in your DAG that needs more than a single deterministic tool call should be routed to whichever of the three actually fits its shape, and you should be able to say why.
- **Self-correction, both scopes:** implement Self-Refine (one draft, one critique against an explicit rubric, one revision) for sub-task outputs that are cheap to redo, and Reflexion (retry the entire task across multiple trials, carrying a capped episodic buffer of verbal reflections from prior failed trials into the next attempt) for the sub-task or request type where a single retry isn't enough and the agent needs to learn across attempts within the same run.
- **Grounded vs. ungrounded critique:** for every critique step in your system, Self-Refine, Reflexion's evaluate step, and LATS's external feedback, state explicitly what the source of truth is. If you can run a test, call a validation tool, check a schema, or check against your real database, do that instead of asking the same model if it's happy with its own output, then test how making an independent critic (a different LLM) would change the evaluation. Show at least one sub-task in your DAG where you deliberately swapped an ungrounded self-critique for a grounded one, and show the failure case the grounded version catches that the ungrounded version missed.

- **Cost and quality comparison across everything, not a subset:** build a test suite of real requests from your chosen problem (include at least one case that should favor decomposition-first over dynamic and vice versa, one that needs lookahead search, and one where a single retry isn't enough and only Reflexion's cross-trial memory helps). Run every method against every applicable case: decomposition-first vs. dynamic decomposition, Plan-and-Solve vs. Tree of Thoughts vs. LATS, Self-Refine vs. Reflexion. Produce one comparison table scoring accuracy/task success, total LLM calls, total tokens, latency. Pick what your system actually ships with, per sub-task type, and justify each choice against the table, not against which method sounds the most sophisticated. Write test cases (prompts for the agent) that would show an actual usage of it, make them plentiful please.



If your project only demonstrates concerns that would work identically on a three-step to-do list, you haven't found the right problem yet. Go back and pick a request with enough real ambiguity, real branching, and real cost of a wrong plan to justify all of it honestly.

YOU MUST BUILD ON TOP OF THE REFERENCE TOOLKIT

This lab is not “implement these algorithms from a blank file.” A working reference implementation of every algorithm above already exists and you are required to build on top of it, not around it:

github.com/AmrSheta22/task_decomposition_and_planning

In the case of writing unnecessary or already implemented functions in the provided repo, the team will be penalized up to 50% of their grade (the team grade). Fork it into your team's org and wire it into your existing repo as the implementation layer for your new agent. Concretely:

- **Decomposition:** use algorithms/decomposition.py and algorithms/dynamic_decomposition.py as your starting point for the DAG concern, adapted to generate and execute sub-tasks against your actual MCP tools and database, not the toolkit's generic demo prompts.
- **Planning:** use algorithms/plan_and_solve.py, algorithms/tree_of_thoughts.py, and algorithms/lats.py as your starting point for the planning concern. You will need to swap the toolkit's default model provider for whatever you're already using elsewhere in your repo, keep the interfaces, don't rebuild the search loops from scratch.
- **Self-correction:** use algorithms/self_refine.py and algorithms/reflexion.py as your starting point for the self-correction concern.

- **Grounding:** the toolkit's algorithms/environment.py ships with a deliberately fake evaluator, a randomized score with no connection to reality, specifically so you replace it. Your job is to implement a real EnvironmentFeedback source: an actual test run, an actual call against your MCP server, an actual check against your database, whatever "did this sub-task actually succeed" means for your company. A LATS or grounded-Reflexion implementation still pointed at the toolkit's randomized default at submission time earns no credit for grounding, regardless of what the code around it looks like.
- **Evidence:** the toolkit already saves a JSON trace per run to artifacts/ with plans, node outputs, critic feedback, episodic memories, MCTS visits, and branch reflections. Use that trace format, extended as needed, as the evidence backing your comparison table and your demo, don't build a second logging system in parallel.
- **Read first:** read the toolkit's own README and its "Suggested exercises" section before you start, several of them (comparing PS against ToT by call count, comparing ToT's self-scores against LATS's environment scores and visit counts, watching dynamic decomposition change course after a failed observation) are close to exactly the evaluation this lab is asking for, and re-deriving them from scratch is wasted effort the toolkit already spent for you.

Don't rebuild the toolkit's search or scheduling logic in a parallel file because it was faster than reading someone else's code. Genuine extension of the toolkit is graded the same way genuine extension of your own mcp_server/ is graded, and duplicated effort reads as such.

A WORKED EXAMPLE, READ THIS, THEN DO SOMETHING DIFFERENT WITH YOUR OWN COMPANY

This shows the shape of the exercise on top of the MCP Server Lab and Memory & RAG Lab examples, not the problem to copy. Extend your own company, your own existing repo, and pick a different agent and a different problem than whatever you built for memory and RAG.

Picture the same "Larkspur Veterinary Clinics" system again. The memory/RAG agent already handles front-desk triage calls and clinical-policy questions. A separate, real pain point shows up in scheduling: twice a week, a vet calls in sick or an emergency case walks in, and the office manager has to re-shuffle the day's appointment board by hand, deciding which appointments move, which get double-booked with a second vet, which get called to reschedule, and in what order, while respecting constraints like "dental cleanings need 45 minutes minimum" and "Dr. Osei doesn't do large-animal sedation." This is a planning problem: it has real branching (several valid reshuffles exist), a real cost to a wrong plan (a double-booked emergency slot), and a real difference between "commit to one plan" and "adjust as new information comes in" (a rescheduled client might not answer the phone, which changes the rest of the plan).

The team builds a Scheduling Agent, separate from the memory/RAG agent, using the toolkit's decomposition and dynamic_decomposition modules to break "reshuffle Tuesday's board" into an ordered set of sub-tasks (identify affected appointments, rank by urgency and reschedulability, propose moves, call the MCP server's real appointment-write tool). They route the "rank by urgency" sub-task, which genuinely benefits from considering several orderings before committing, through Tree of Thoughts, and route the final "propose the reshuffle" sub-task through LATS, with environment.py replaced by a real check: does the proposed reshuffle actually pass their existing scheduling-conflict validator. A first attempt that produces a double-booking gets a Reflexion-style verbal reflection ("I didn't account for the 45-minute minimum on dental slots") carried into the next trial.

Top-level decomposition: reshuffling Tuesday's board (20 real cases)

Method	Task success	Avg. LLM calls	Avg. tokens	Avg. latency	Est. cost/run
Decomposition-first	14/20	1 plan + 4 nodes	6,100	3.1s	$0.04
Dynamic decomposition	17/20	~7 (varies)	8,900	5.4s	$0.06

Dynamic decomposition beats decomposition-first specifically because reschedule attempts genuinely fail mid-plan (a client doesn't pick up), and only dynamic decomposition can react instead of executing a stale plan; the team ships dynamic decomposition as the default for the top-level reshuffle and keeps decomposition-first only for the fully mechanical sub-tasks with no real branching.

Planning the "rank by urgency" and "propose the reshuffle" sub-tasks (15 cases each)

Method	Sub-task success	Avg. LLM calls	Avg. tokens	Avg. latency	Est. cost/run
Plan-and-Solve (ranking)	11/15	1	1,400	0.9s	$0.01
Tree of Thoughts (ranking)	14/15	9	5,200	3.8s	$0.04
LATS, ungrounded env. (toolkit default)	9/15	11	7,600	6.2s	$0.06
LATS, grounded env. (real conflict validator)	14/15	13	8,300	6.9s	$0.07

Tree of Thoughts clearly beats Plan-and-Solve on the ranking sub-task for a small extra cost, worth paying. LATS with the toolkit's default randomized environment barely outperforms doing nothing, confirming the guardrail below, an ungrounded LATS is expensive theater; once wired to the real conflict validator, LATS earns its cost specifically on the final proposal step, where a wrong plan is expensive to unwind by phone. The table, not a guess, is what the README cites as the justification.

WHAT TO BUILD AND SUBMIT

- **An updated README:** what planning problem you found on top of your existing system, why it's real, which agent owns it, and how each concern above shows up in your solution.
- **A planning/ folder (forked/adapted from the toolkit, credited as such):** your decomposition-first and dynamic decomposition implementations wired into your real MCP tools, your Plan-and-Solve/Tree of Thoughts/LATS implementations, your Self-Refine/Reflexion implementations, and your real EnvironmentFeedback source replacing the toolkit's randomized default.
- **A planning_eval/ folder:** your real-request test suite, the script or notebook that ran every method against it, and the artifacts/ traces backing your comparison table.
- **Any changes to mcp_server/ or agent/** needed to stand up the new agent and route its sub-tasks to the right planning method. This should visibly reuse the existing server and database, and should not touch or duplicate the memory/RAG agent's code path.
- **Locatable concerns:** inside planning/, every concern should be easy for a grader to locate without reading the whole file: the DAG construction and cycle check, the decomposition-first vs. dynamic branch point, the routing logic that decides which sub-task goes to PS vs. ToT vs. LATS, the grounded environment implementation, and the Self-Refine/Reflexion critique code.
- **The full comparison table,** embedded in the README with the numbers that drove your final per-sub-task method choice.
- **A short demo transcript or recording** showing: a real request decomposed both ways with the divergence visible, at least one sub-task solved by each of PS, ToT, and LATS, a Self-Refine revision, a Reflexion run that carries a reflection across trials, and the grounded environment catching a failure the ungrounded default would have missed.
- **Commit history** should show real contributions from all team members, and should show commits into the forked toolkit code, not just around it.

RESOURCES

- **The reference toolkit (required):** github.com/AmrSheta22/task_decomposition_and_planning, read algorithms/*.py and the README's own “Suggested exercises” before writing new code.
- **Plan-and-Solve Prompting** (Wang et al., ACL 2023) for the original plan/solve two-phase prompt.
- **Tree of Thoughts** (Yao et al., 2023) for the generate/evaluate/search loop and the Game-of-24 worked example the toolkit's tree_of_thoughts.py is modeled on.
- **Language Agent Tree Search / LATS** for the MCTS four-phase loop (select, expand and simulate, evaluate/reflect, backpropagate) the toolkit's lats.py implements.

- **Self-Refine** (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) for the single-draft critique loop versus the multi-trial verbal-reinforcement loop.

A FEW GUARDRAILS

- Pick a planning problem where a wrong plan actually costs something. If a bad decomposition or an unpruned bad branch changes nothing, you haven't found the right problem yet.
- Decomposition-first and dynamic decomposition must both run against the same real request type. Implementing only one and describing the other in prose earns no credit for the concern.
- LATS and Reflexion's critique must be genuinely grounded for at least the sub-task type where you claim grounding matters. An environment.py still returning the toolkit's randomized default at submission time is an ungrounded system regardless of what the surrounding code implies.
- The comparison table must cover every required method, not a convenient subset. A table missing Plan-and-Solve, or missing dynamic decomposition, or missing the ungrounded-vs-grounded LATS contrast, doesn't satisfy the concern even if the methods that are present look thorough.
- Don't rebuild the toolkit's algorithms from scratch, and don't rebuild your existing database, MCP server, or memory/RAG agent from scratch either. This lab is graded on genuine extension of existing work in both directions.
- Never commit an API key or database credential. Use a .env file and confirm it's still in .gitignore.
- Keep your planning test suite fixed once you start evaluating. Changing test cases between runs invalidates your comparison table.

WORKING AS A TEAM: ISSUES AND MODULAR WORK

Every GitHub Issue is graded on its rationale, not its existence. An issue that says “add Tree of Thoughts” earns nothing. One that states the problem, the constraint, and the acceptance criteria does.

Breaking the project into modular parts: split work across the decomposition concern (both methods), the planning-algorithm concern (PS/ToT/LATS and routing between them), the self-correction concern (Self-Refine/Reflexion and the grounded environment), the evaluation harness, and the integration touching agent/ and mcp_server/. Each concern gets its own owner before any code is written.

Writing an issue with real rationale: for example, “office manager re-plans Tuesday's board by hand every time a vet calls in sick, and a wrong reshuffle double-books an emergency slot” rather than “add scheduling agent.” Name the constraint that makes the concern genuinely necessary in your system, and write acceptance criteria a teammate could check without asking what you meant.

Assigning, linking, and closing issues: one owner per issue, referenced from the pull request that resolves it (“Closes #-”), with a short closing comment explaining what changed. An issue opened and closed with no linked commit or discussion won't count as attributable teamwork.

GRADING RUBRIC

Your team deliverable is scored out of 100 fixed points. Every row is binary, full points only when its criteria are genuinely met, not partially. Your final individual grade is 70% this team score plus 30% an individual contribution score built from Git history and issue rationale.

Category	Points	What's assessed
Problem framing and suitability	10	A genuine planning problem on a different agent than your memory/RAG agent, originality vs. the worked example, whether it truly needs decomposition and search rather than a single tool call.

Category	Points	What's assessed
Extending the existing system and toolkit	8	The existing mcp_server/, db/, and the reference toolkit are all visibly reused and forked into, not duplicated or rebuilt from scratch.
Task decomposition, both methods	15	Decomposition-first and dynamic/interleaved decomposition both correctly implemented against the same real request type, acyclicity enforced, a real case where the two methods diverge.
Planning algorithms, all three	20	Plan-and-Solve, Tree of Thoughts, and LATS all correctly implemented and routed to sub-tasks whose shape genuinely fits each, using the toolkit as the implementation base.
Self-correction, both scopes	12	Self-Refine and Reflexion both correctly implemented, Reflexion's episodic buffer genuinely carried across trials.
Grounded environment for LATS/Reflexion	10	The toolkit's randomized default replaced with a real external feedback source, with a shown case the grounded version catches that an ungrounded self-critique would have missed.
Full cost and quality comparison table	10	Every required method compared on accuracy, calls, tokens, latency, and cost against a real fixed test suite, with per-sub-task method choices justified by the table.
Repository usability and safety	5	Clear organization, demo evidence covering every concern, reproducible setup, no exposed secrets.
Teamwork and issue rationale	5	Issues with genuine rationale, single ownership, and traceable resolution through linked pull requests.
Agent and system integration	5	The new agent is genuinely wired into the live MCP server/agent loop alongside, not instead of, the memory/RAG agent, with complete run instructions.
Total	**100**	